
Bot Discord

Państwa – Miasta

Dokumentacja techniczna

Autor: Kamil Bujak

E-mail: kamil.bujak04@gmail.com

Rok: 2025/2026

Spis treści

1	Architektura systemu	2
1.1	Przegląd	2
1.2	Struktura plików	2
2	Opis modułów	4
2.1	main.py — punkt wejścia	4
2.2	game_session.py — logika gry	4
2.3	round.py — runda	6
2.4	validator.py — walidacja hybrydowa	6
2.5	database.py — baza danych	7
2.5.1	Modele SQLAlchemy 2.0	7
2.5.2	Schemat bazy danych	8
2.6	voice_manager.py — ogłoszenia głosowe	8
3	Konfiguracja i wdrożenie	10
3.1	Zmienne środowiskowe	10
3.2	Parametry gry	10
3.3	Logowanie	10
3.4	Uruchomienie produkcyjne (Linux/systemd)	11
4	Obsługa błędów i bezpieczeństwo	12
4.1	Obsługa błędów	12
4.2	Bezpieczeństwo	12
A	Słownik pojęć	13

1. Architektura systemu

1.1 Przegląd

System składa się z czterech warstw:

1. **Warstwa Discord** — discord.py 2.x: komendy, embedy, role, DM, voice, UI buttons.
2. **Warstwa logiki gry** — sesje, rundy, lobby, punktacja, faza zaskarżania.
3. **Warstwa walidacji** — lokalne listy słów (primary) + zewnętrzne API (fallback).
4. **Warstwa danych** — SQLite przez SQLAlchemy 2.0 ORM.

1.2 Struktura plików

```
1 DISCORD BOT/  
2 |-- bot/  
3 |   |-- main.py           # punkt wejścia, komendy, reset  
   |   |   sezonowy  
4 |   |-- game_session.py   # logika gry: lobby, rundy, DM,  
   |   |   zaskarżanie  
5 |   |-- round.py          # pojedyncza runda, punktacja  
6 |   |-- player.py         # obiekt gracza  
7 |   |-- validator.py      # walidacja: lokalne listy + API  
   |   |   fallback  
8 |   |-- countries_pl.py   # ~250 polskich nazw państw (  
   |   |   primary)  
9 |   |-- cities_pl.py      # ~1155 miast świata po polsku (  
   |   |   primary)  
10 |   |-- names_pl.py      # polskie imiona + zdrobnienia  
11 |   |-- animals_pl.py    # polskie nazwy zwierząt (primary)  
12 |   |-- plants_pl.py     # polskie nazwy roślin (primary)  
13 |   |-- database.py      # modele SQLAlchemy 2.0,  
   |   |   DatabaseHandler
```

```
14 |   |-- stats_manager.py   # role Discord Top 1/2/3
15 |   '-- voice_manager.py   # TTS: gTTS (primary) + pyttsx3 (
      fallback offline)
16 |-- bot.db                # SQLite (generowana automatycznie
      )
17 |-- bot.log               # logi (generowany automatycznie)
18 |-- .env                  # BOT_TOKEN (nie commitowac!)
19 |-- .gitignore
20 |-- requirements.txt
21 |-- dokumentacja_uzytkownika.tex
22 |-- dokumentacja_techiczna.tex
23 |-- dokumentacja_api.tex
24 |-- diagram_aktywnosci.png
25 '-- diagram_aktywnosci2.png
```

Listing 1.1: Drzewo katalogów projektu

2. Opis modułów

2.1 main.py — punkt wejścia

Inicjalizuje bota, rejestruje wszystkie komendy i uruchamia zadanie cykliczne sprawdzające reset sezonu.

Kluczowe elementy:

- `commands.Bot` z prefiksem `/` i intencjami `message_content` + `members`.
- `logging.basicConfig` — logi zapisywane do `bot.log` i konsoli.
- Zadanie `season_reset_check` (pętla `tasks.loop(hours=24)`) — sprawdza co 24h czy minęło 10 dni od ostatniego resetu.
- Słownik `sessions: dict[int, GameSession]` — aktywne sesje per serwer.

2.2 game_session.py — logika gry

Stany sesji:

- `waiting` — lobby otwarte (domyślnie 30 s), gracze dołączają przez `/dołącz`.
- `active` — gra trwa, bot wysyła DM i zbiera odpowiedzi.
- `finished` — gra zakończona, sesja gotowa do usunięcia.

Dołączanie do voice: Bot dołącza do kanału głosowego *natychmiast po wpisaniu `/start`* i ogłasza odliczanie do startu gry.

```
1 if self.ctx.author.voice:
2     await self.voice.join_channel(self.ctx.author.voice.
        channel)
3     await self.voice.announce(
4         f"Gra rozpocznie sie za {LOBBY_TIME} sekund. Dolacz!"
        ")
5     await asyncio.sleep(LOBBY_TIME)
```

Listing 2.1: Dołączenie do voice podczas lobby

Kolejność zdarzeń w rundzie:

1. Bot **ogłasza głosowo** treść rundy (litera, czas) i czeka aż skończy.
2. Dopiero po wypowiedzeniu: embed na kanale i DM do graczy — jednocześnie.
3. Proporcjonalne przypomnienia (50% i ~15% czasu) — głos + DM równolegle.
4. Runda kończy się natychmiast gdy **wszyscy** gracze wyślą odpowiedzi.

Proporcjonalne ostrzeżenia:

```

1 warn1_remaining = ROUND_TIME // 2           # 50% czasu
2 warn2_remaining = max(10, ROUND_TIME // 6) # ~15% czasu (min
    10s)
3 # Dla 60s: ostrzeżenia po 30s i 50s
4 # Dla 300s: ostrzeżenia po 150s i 250s

```

Listing 2.2: Obliczanie czasów przypomnień

Faza zaskarżania (Discord UI Buttons): Po każdej rundzie uruchamiana jest faza zaskarżania z interaktywnymi przyciskami:

1. **AppealView** (60 s) — każdy gracz może kliknąć przycisk przy swojej *odrzuconej* odpowiedzi. Tylko właściciel może kliknąć swój przycisk.
2. **VoteView** (30 s na każdą zaskarżoną odpowiedź) — pozostali gracze głosują przyciskami **Akceptuj** / **Odrzuć**. Gracz nie może głosować na własną odpowiedź.
3. Jeśli **ponad połowa** głosów jest za akceptacją — gracz otrzymuje **+10 pkt** bonusu.
4. **NextRoundView** — przycisk „*Następna runda*” widoczny tylko dla organizatora gry (autora `/start`). W ostatniej rundzie przycisk zmienia się na „*Zakończ rozgrywkę*” (czerwony).

Embedy Discord: Wszystkie wiadomości bota są kolorowymi kartami (`discord.Embed`):

- Niebieski (RGB 88,101,242) — informacje o grze i rundach.
- Zielony — potwierdzenie startu gry, zaakceptowane zaskarżenia.
- Czerwony — błędy, brak odpowiedzi, odrzucone zaskarżenia.
- Żółty — wyniki końcowe i zwycięzca.
- Żółty — faza zaskarżania i głosowanie.

2.3 round.py — runda

Przechowuje odpowiedzi w strukturze:

```
1 # {discord_id: {kategoria: (odpowiedz, is_valid, powod)}}
2 validated_answers: dict[str, dict[str, tuple[str, bool, str]
  ]]
```

Listing 2.3: Struktura danych rundy

Algorytm punktacji (calculate_points):

1. Dla każdej kategorii grupuje poprawne odpowiedzi po treści (po normalizacji).
2. Unikalna odpowiedź → 10 pkt; zduplikowana → 5 pkt.
3. Niepoprawna lub brak → 0 pkt.

2.4 validator.py — walidacja hybrydowa

Architektura walidacji — dwie warstwy:

1. **Lokalna lista** (primary) — zbiory `set` w plikach `*_pl.py`, sprawdzane jako pierwsze. Odpowiedź znaleziona na liście jest akceptowana natychmiast, bez żadnego wywołania API.
2. **Zewnętrzne API** (fallback) — wywoływane tylko gdy odpowiedzi *nie ma* na lokalnej liście. Timeout 5 sekund; błąd lub timeout → **False** (nigdy fail-open).

Kategoria	Lokalna lista	API fallback	Uwagi
Państwo	<code>countries_pl.py</code> ~250	REST Countries v3.1	Dokładne dopasowanie polskiej nazwy
Miasto	<code>cities_pl.py</code> ~1155	Nominatim (OSM)	Dokładne dopasowanie, max 1 req/s
Imię	<code>names_pl.py</code> + zdrobnienia	brak API	Tylko lista (brak pol. API imion)
Zwierzę	<code>animals_pl.py</code>	Polska Wikipedia	Sprawdza kategorie: Ssaki, Ptaki...
Roślina	<code>plants_pl.py</code>	Polska Wikipedia	Sprawdza kategorie: Rośliny, Drzewa...

Tabela 2.1: Źródła walidacji według kategorii

Weryfikacja odpowiedzi przez API:

- **REST Countries** — odpowiedź akceptowana tylko przy *dokładnym* dopasowaniu polskiej nazwy zwróconego kraju (pole `translations.pol.common`).
- **Nominatim** — sprawdza pola `name`, `name:pl`, `alt_name` i `display_name`; wymagane dokładne dopasowanie do co najmniej jednego z nich.
- **Wikipedia** — pobiera kategorie artykułu; akceptuje jeśli kategoria zawiera słowo kluczowe (np. *ssaki*, *rośliny*). Przekierowania Wikipedii są obsługiwane — Panda → Panda wielka → kategoria Ssaki .

Normalizacja (`_norm`):

```

1 def _norm(text: str) -> str:
2     # 'ł' zastępuje 'l'      l nie dekompozycjonuje sie przez
        NFKD!
3     text = text.lower().replace("ł", "l")
4     return (
5         unicodedata.normalize("NFKD", text)
6         .encode("ascii", "ignore")
7         .decode("ascii")
8     )
9     # Efekt: "Krakow" == "Krakow", "Pawel" == "Pawel"

```

Listing 2.4: Funkcja normalizacji

Cache między rundami: Wyniki walidacji są cache'owane w słowniku `_cache` z kluczem (kategoria, `norm(odpowiedź)`, `norm(litera)`). To samo słowo jest walidowane przez API tylko raz — kolejne wystąpienia pobierane są z cache.

2.5 database.py — baza danych

2.5.1 Modele SQLAlchemy 2.0

```

1 class PlayerModel(Base):
2     discord_id : Mapped[str] = mapped_column(String,
        primary_key=True)
3     guild_id   : Mapped[str] = mapped_column(String,
        primary_key=True)
4     username   : Mapped[str]
5     total_points : Mapped[int]
6     games_played : Mapped[int]
7     games_won   : Mapped[int]
8
9 class GameModel(Base):

```



```

10     id          : Mapped[int]
11     guild_id   : Mapped[str]
12     played_at  : Mapped[datetime]
13     winner_id  : Mapped[str | None]
14     rounds     : Mapped[int]
15
16     class ResetLog(Base):
17         id      : Mapped[int]
18         reset_at : Mapped[datetime]

```

Listing 2.5: Modele tabel

Użyto nowej składni `Mapped[]` z SQLAlchemy 2.0, która daje poprawne typy statyczne (Pylance nie widzi już błędów `Column[int]`).

2.5.2 Schemat bazy danych

Tabela	Kolumna	Typ	Opis
players	discord_id	TEXT (PK)	ID użytkownika Discord
	guild_id	TEXT (PK)	ID serwera Discord
	username	TEXT	Wyświetlana nazwa
	total_points	INT	Punkty w sezonie
	games_played	INT	Liczba gier
	games_won	INT	Liczba wygranych
games	id	INT (PK)	Autoinkrementowany klucz
	guild_id	TEXT	ID serwera
	played_at	DATETIME	Czas zakończenia
	winner_id	TEXT	ID zwycięzcy
	rounds	INT	Liczba rund
reset_log	id	INT (PK)	Autoinkrementowany klucz
	reset_at	DATETIME	Czas resetu sezonu

Tabela 2.2: Schemat bazy danych SQLite

2.6 voice_manager.py — ogłoszenia głosowe

Bot używa dwupoziomowego systemu TTS:

1. **gTTS** (Google Text-to-Speech) — wysoka jakość, język polski, timeout 6 sekund. Wymaga internetu.
2. **pyttsx3** (fallback) — lokalny Windows SAPI5, działa offline. Używany automatycznie gdy gTTS jest niedostępne.

Wywołanie `gTTS.save()` jest blokujące (I/O sieciowe) i uruchamiane w osobnym wątku przez `asyncio.run_in_executor`, aby nie blokować pętli zdarzeń `asyncio`.

```
1  async def announce(self, text: str) -> None:
2      try:
3          # gTTS w osobnym watku (timeout 6s)
4          await asyncio.wait_for(
5              loop.run_in_executor(None, _gtts_save, text,
6                                  path),
7              timeout=6,
8          )
9      except Exception:
10         # Fallback: lokalny pyttsx3
11         await loop.run_in_executor(None, _pyttsx3_save, text
12                                     , path)
13     await self._play(path, loop)
```

Listing 2.6: Logika wyboru silnika TTS

3. Konfiguracja i wdrożenie

3.1 Zmienne środowiskowe

Zmienna	Opis	Przykład
BOT_TOKEN	Token bota Discord (wymagany)	MTQ4Nj...
DATABASE_URL	Ścieżka do SQLite (opcjonalna)	sqlite:///bot.db

Tabela 3.1: Zmienne środowiskowe (.env)

3.2 Parametry gry

Stała	Domyślna	Komenda zmiany
MAX_ROUNDS	5	/ustawienia rundy N
ROUND_TIME	60s	/ustawienia czas N
LOBBY_TIME	30s	/ustawienia lobby N

Tabela 3.2: Parametry gry zmieniane w trakcie działania

Komenda /ustawienia obsługuje **wiele parametrów naraz**:

```
1 /ustawienia rundy 5 czas 60 lobby 30
```

Listing 3.1: Zmiana wielu parametrów jedną komendą

Kolejność parametrów nie ma znaczenia; można podać dowolną kombinację.

3.3 Logowanie

Bot zapisuje logi do bot.log (plik w katalogu DISCORD BOT) oraz wyświetla je w konsoli. Wyciszone są logi bibliotek discord i aiohttp (poziom WARNING), widoczne są tylko logi bota (poziom INFO i wyżej).

3.4 Uruchomienie produkcyjne (Linux/systemd)

```
1 [Unit]
2 Description=Discord Bot Państwa-Miasta
3 After=network.target
4
5 [Service]
6 WorkingDirectory=/opt/discord-bot
7 ExecStart=/usr/bin/python3 bot/main.py
8 Restart=on-failure
9 EnvironmentFile=/opt/discord-bot/.env
10
11 [Install]
12 WantedBy=multi-user.target
```

Listing 3.2: Plik jednostki systemd

4. Obsługa błędów i bezpieczeństwo

4.1 Obsługa błędów

Sytuacja	Zachowanie
Komenda poza serwerem	<code>@commands.guild_only()</code> — odrzucenie
Brak uprawnień admina	Komunikat błędu, komenda nie wykonana
Gracz zablokował DM	Ostrzeżenie na kanale, gracz pomijany
Brak <code>BOT_TOKEN</code>	<code>RuntimeError</code> z czytelnym komunikatem
Nieznana komenda	Sugestia użycia <code>/pomoc</code>

Tabela 4.1: Obsługa wyjątków

4.2 Bezpieczeństwo

- Plik `.env` zawiera token bota i jest w `.gitignore`.
- Plik `bot.db` jest w `.gitignore` (dane użytkowników).
- Plik `bot.log` jest w `.gitignore`.
- Komendy admina wymagają uprawnienia `administrator` na serwerze Discord.

A. Słownik pojęć

DM

Direct Message — prywatna wiadomość Discord.

Guild

Serwer Discord — wspólna przestrzeń użytkowników.

Intencje (Intents)

Konfiguracja jakie zdarzenia bot odbiera od Discord.

Lobby

Faza oczekiwania, w której gracze mogą dołączyć (/dołącz).

Sezon

Okres między automatycznymi resetami rankingu (10 dni).

ORM

Object-Relational Mapper — biblioteka mapująca obiekty Python na tabele SQL.

TTS

Text-to-Speech — zamiana tekstu na mowę (gTTS primary, pyttsx3 fallback, język polski).

Normalizacja

Usunięcie ogonków i zmiana na małe litery: *Łódź* → *lodz*.